

TP1 Python — Partie 1

Corrigé expliqué pas à pas des six exercices du notebook

Ce document accompagne le notebook **TP1python_partie1.ipynb**. Pour chaque exercice, il donne le code de la solution, l'explication du raisonnement, le résultat d'une exécution réelle et les pièges à connaître. Tout le code a été exécuté et vérifié, y compris sur des cas limites.

Vue d'ensemble

N°	Exercice	Notions travaillées
1	Système de notation pondérée	<code>input()</code> , conversions <code>float()</code> , <code>int()</code> , <code>str()</code>
2	Analyse d'une équation du second degré	<code>if / elif / else</code> , tests imbriqués, booléen
3	Contrôle de saisie et boucle d'affichage	boucle <code>while</code> , boucle <code>for</code> , <code>range()</code>
4	Traitement des notes	fonctions, paramètre par défaut, immuabilité, <code>map()</code> , <code>zip()</code>
5	Saisie ordonnée et médiane	listes, indices, boucles imbriquées, parité
6	Configuration d'un système multi-agents	dictionnaires, listes et tuples imbriqués

À savoir avant de lire

- La fonction `input()` renvoie **toujours** une chaîne de caractères. Presque tous les exercices commencent donc par une conversion.
- Le notebook d'origine ne contenait pas de cellule de code pour l'exercice 2 (équation) : elle a été ajoutée juste après son énoncé.
- Les sorties enregistrées dans le notebook proviennent d'une exécution avec les saisies montrées dans ce document. En relançant les cellules, vous pouvez saisir vos propres valeurs.

Environnement de travail : uv

Le projet est géré avec **uv**. Le fichier `pyproject.toml` déclare les dépendances (`notebook` et `ipykernel`) et `uv.lock` fige leurs versions exactes, ce qui rend l'environnement reproductible sur n'importe quelle machine.

```
uv init --bare          # crée pyproject.toml
uv add notebook ipykernel # ajoute les dépendances, crée .venv et uv.lock
uv sync                 # recrée l'environnement à partir de uv.lock
uv run jupyter notebook TP1python_partiel.ipynb # lance le notebook
```

- `uv add` installe les paquets dans un environnement virtuel `.venv` propre au projet, sans toucher au Python du système.
- `uv run` exécute une commande dans cet environnement, sans avoir à l'activer à la main.
- Le dossier `.venv` n'est pas versionné (il est dans `.gitignore`) : `uv sync` suffit à le reconstruire.

Exercice 1 — Système de notation pondérée

Notions : saisie clavier, conversions de types (casting), concaténation

But. Lire un identifiant, deux notes et deux coefficients, calculer la moyenne pondérée, puis l'afficher dans une phrase complète construite avec `str()`.

Solution

```
# === EXERCICE 1 : Système de notation pondérée ===

# 1. Identifiant de l'étudiant : on le garde tel quel, en chaîne de caractères (str)
identifiant = input("Identifiant de l'étudiant (ex : CNE123456) : ")

# 2. Notes : input() renvoie un str, on convertit immédiatement en float
note_projet = float(input("Note du projet pratique (TP) : "))
note_ecrit = float(input("Note de l'examen écrit : "))

# 3. Coefficients : conversion en entiers (int)
coeff_projet = int(input("Coefficient du projet : "))
coeff_ecrit = int(input("Coefficient de l'écrit : "))

# 4. Moyenne pondérée
moyenne = (note_projet * coeff_projet + note_ecrit * coeff_ecrit) / (coeff_projet + coeff_ecrit)

# 5. Affichage : str() convertit le float en texte pour pouvoir le concaténer avec +
message = ("Pour l'étudiant " + identifiant
          + ", la moyenne finale du module est de " + str(moyenne) + " / 20.")
print(message)
```

Explication

- **L'identifiant reste un str.** « CNE123456 » contient des lettres : ce n'est pas un nombre, on ne fait aucun calcul dessus, donc aucune conversion.
- **Les notes en float.** Une note peut être décimale (14.5). On enveloppe directement `input()` dans `float()` : la conversion se fait « immédiatement », comme le demande l'énoncé. Sans conversion, "17" * 2 donnerait "1717" (répétition de texte) et non 34.
- **Les coefficients en int.** Ce sont des entiers. Attention : `int("2.5")` provoque une erreur `ValueError`, car `int()` n'accepte qu'un texte représentant un entier.
- **La formule.** Chaque note est multipliée par son coefficient, puis on divise par la somme des coefficients. Les parenthèses sont indispensables : sans elles, la division `/` serait effectuée avant les additions. En Python 3, `/` renvoie toujours un `float`.
- **L'affichage avec str().** L'opérateur `+` ne sait pas coller du texte et un nombre : `"moyenne : " + 15.5` lève une `TypeError`. On convertit donc la moyenne en texte avec `str(moyenne)` avant de concaténer.

Exécution

```
Identifiant de l'étudiant (ex : CNE123456) : CNE123456
Note du projet pratique (TP) : 17
Note de l'examen écrit : 14.5
Coefficient du projet : 2
Coefficient de l'écrit : 3
Pour l'étudiant CNE123456, la moyenne finale du module est de 15.5 / 20.
```

Vérification à la main : $(17 \times 2 + 14,5 \times 3) / (2 + 3) = (34 + 43,5) / 5 = 77,5 / 5 = \mathbf{15,5}$. C'est exactement la phrase attendue par l'énoncé.

Piège. Si la moyenne ne tombe pas juste (par exemple 13.666666666666666), on peut l'arrondir avant l'affichage avec `str(round(moyenne, 2))`.

Exercice 2 — Analyse d'une équation $ax^2 + bx + c = 0$

Notions : structures conditionnelles, tests imbriqués, variable booléenne

But. Déterminer la nature de l'équation, puis, si elle est bien du second degré, le nombre de ses solutions réelles à partir du discriminant $\Delta = b^2 - 4ac$. L'énoncé impose une architecture en **deux blocs séparés**.

Solution

```
# === EXERCICE 2 : Analyse d'une équation  $ax^2 + bx + c = 0$  ===

a = float(input("Coefficient a : "))
b = float(input("Coefficient b : "))
c = float(input("Coefficient c : "))

# --- Première partie : tests imbriqués ---
if a == 0:
    # a est nul : ce n'est plus une équation du second degré
    if b == 0:
        print("Équation dégénérée : a = 0 et b = 0, il ne reste que c = 0.")
    else:
        print("Équation du premier degré :  $bx + c = 0$ , solution x =", -c / b)
    est_second_degre = False
else:
    est_second_degre = True

# --- Deuxième partie : structure conditionnelle séparée (pas imbriquée dans la première) ---
if est_second_degre:
    delta = b ** 2 - 4 * a * c
    print("Discriminant  $\Delta$  =", delta)

    if delta > 0:
        x1 = (-b - delta ** 0.5) / (2 * a)
        x2 = (-b + delta ** 0.5) / (2 * a)
        print(" $\Delta > 0$  : l'équation admet 2 solutions réelles distinctes :", x1, "et", x2)
    elif delta == 0:
        x0 = -b / (2 * a)
        print(" $\Delta = 0$  : l'équation admet 1 solution réelle (double) :", x0)
    else:
        print(" $\Delta < 0$  : l'équation n'admet aucune solution réelle.")
```

Explication

- **Première partie, tests imbriqués.** Le `if a == 0` extérieur contient un second `if b == 0` : c'est cela, un test imbriqué. Si $a = 0$ et $b = 0$, il ne reste que « $c = 0$ » : l'équation est dégénérée (toujours vraie si $c = 0$, impossible sinon). Si $a = 0$ mais $b \neq 0$, on obtient $bx + c = 0$, une équation du premier degré dont la solution est $x = -c / b$.
- **Position de `est_second_degre = False`.** Cette ligne est alignée avec le `if b == 0` intérieur, donc elle appartient au bloc « $a = 0$ » et s'exécute dans les deux sous-cas. Une seule ligne suffit au lieu de la répéter deux fois.
- **Deuxième partie, bloc séparé.** Le `if est_second_degre:` commence tout à gauche, sans indentation : il est **à la suite** du premier bloc et non à l'intérieur. Le booléen sert de « message » transmis du premier bloc au second.
- **Le discriminant.** `b ** 2` calcule b^2 . Selon le signe de Δ : deux solutions si $\Delta > 0$, une solution double si $\Delta = 0$, aucune solution réelle si $\Delta < 0$.
- **Bonus.** Le programme affiche aussi les solutions. `delta ** 0.5` est la racine carrée de Δ , ce qui évite d'importer le module `math`.

Exécution

```
Coefficient a : 1
Coefficient b : -3
Coefficient c : 2
Discriminant  $\Delta = 1.0$ 
 $\Delta > 0$  : l'équation admet 2 solutions réelles distinctes : 1.0 et 2.0
```

Tous les cas testés

a	b	c	Résultat affiché
0	0	5	Équation dégénérée
0	2	-4	Équation du premier degré, $x = 2.0$
1	-3	2	$\Delta = 1 > 0$: 2 solutions réelles (1.0 et 2.0)
1	2	1	$\Delta = 0$: 1 solution double (-1.0)
1	1	1	$\Delta = -3 < 0$: aucune solution réelle

Piège. Ne pas confondre `=` (affectation) et `==` (comparaison). Écrire `if a = 0` : est une erreur de syntaxe.

Exercice 3 — Contrôle de saisie et boucle d'affichage

Notions : boucle `while` (validation), boucle `for`, fonction `range()`

But. Redemander une saisie tant que l'utilisateur ne fournit pas un entier impair, puis afficher tous les entiers de 0 jusqu'à ce nombre **inclus**.

Solution

```
# === EXERCICE 3 : Contrôle de saisie et boucle d'affichage ===

# On convertit en float pour pouvoir détecter aussi bien un décimal qu'un pair
nombre = float(input("Veuillez saisir un nombre entier impair : "))

# 1. Validation : on boucle TANT QUE la saisie est décimale OU paire
while nombre != int(nombre) or int(nombre) % 2 == 0:
    if nombre != int(nombre):
        erreur = "Erreur : Vous avez saisi un nombre décimal."
    else:
        erreur = "Erreur : " + str(int(nombre)) + " est un nombre pair."
    nombre = float(input(erreur + " Veuillez réessayer : "))

# Ici la saisie est forcément un entier impair : on peut la convertir en int
nombre = int(nombre)
print(f"Saisie acceptée ! Voici les nombres de 0 à {nombre} :")

# 2. Séquence : la borne de fin de range() est exclue, d'où le "+ pas" pour inclure nombre
# (pas = -1 si l'utilisateur a saisi un impair négatif, pour compter à rebours)
pas = 1 if nombre > 0 else -1
for i in range(0, nombre + pas, pas):
    print(i)
```

Explication

- **Pourquoi `float()` et pas `int()` ?** Si l'utilisateur tape 3.5, `int("3.5")` plante immédiatement. En convertissant en `float`, on accepte la saisie et c'est le programme qui décide si elle est valide.

- **Détecter un décimal.** `int(nombre)` supprime la partie décimale : `int(3.5)` vaut 3. Si `nombre != int(nombre)`, c'est que le nombre avait une partie décimale.
- **Détecter un pair.** `%` donne le reste de la division : un nombre est pair quand `nombre % 2 == 0`.
- **La condition du `while`.** On reste dans la boucle tant que la saisie est décimale **ou** paire. Dès que les deux tests sont faux, on a un entier impair et la boucle s'arrête.
- **Pas de boucle infinie.** La dernière ligne de la boucle redemande une saisie : la variable `nombre` change à chaque tour, donc la condition finit par devenir fausse.
- **Le message d'erreur sert de question.** Le texte de l'erreur est passé à `input()`, ce qui reproduit exactement le déroulement de l'énoncé (« Erreur : 4 est un nombre pair. Veuillez réessayer : »).
- **La borne de `range()`.** La valeur de fin est toujours exclue : `range(0, 5)` s'arrête à 4. Pour inclure le nombre saisi, on écrit `nombre + 1`. La variable `pas` gère en plus le cas d'un impair négatif (`-3` donne `0, -1, -2, -3`) ; pour un nombre positif, elle vaut simplement 1.

Exécution

```

Veuillez saisir un nombre entier impair : 4
Erreur : 4 est un nombre pair. Veuillez réessayer : 3.5
Erreur : Vous avez saisi un nombre décimal. Veuillez réessayer : 5
Saisie acceptée ! Voici les nombres de 0 à 5 :
0
1
2
3
4
5

```

Limite. Si l'utilisateur tape du texte (« abc »), `float()` lève une `ValueError`. Gérer ce cas demande `try / except`, qui ne fait pas partie de cette séance.

Exercice 4 — Traitement des notes

Notions : fonctions, paramètre par défaut, immuabilité, `map()`, `zip()`

But. Écrire une fonction avec un paramètre par défaut, prouver qu'elle ne modifie pas la variable passée en argument, puis l'appliquer à une liste avec `map()` et associer deux listes avec `zip()`.

Solution

```
# === EXERCICE 4 : Traitement des notes ===

# 1. Paramètre par défaut & immuabilité
def ajouter_bonus(note, bonus=2):
    note = note + bonus    # crée un NOUVEL objet local : la variable extérieure n'est pas touchée
    return note

x = 10
resultat = ajouter_bonus(x)    # bonus n'est pas fourni -> valeur par défaut 2
print("Valeur retournée par la fonction :", resultat)
print("Valeur de x après l'appel :", x, "-> x est resté intact")

# 2. map() : applique ajouter_bonus à chaque note de la liste
notes = [12.0, 15.5]
nouvelles_notes = list(map(ajouter_bonus, notes))
print("Notes initiales :", notes)
print("Notes avec bonus :", nouvelles_notes)

# 3. zip() : parcourt les deux listes en parallèle
etudiants = ["Ali", "Sara"]
for etudiant, note in zip(etudiants, nouvelles_notes):
    print(f"{etudiant} : {note} / 20")
```

Explication

- **Paramètre par défaut.** Dans `def ajouter_bonus(note, bonus=2)`, `bonus` vaut 2 si on ne le précise pas. `ajouter_bonus(10)` renvoie 12, `ajouter_bonus(10, 5)` renverrait 15.
- **Immuabilité.** Un `int` ne peut pas être modifié sur place. `note = note + bonus` fabrique un nouvel objet (12) et l'attache au nom **local** `note`. À l'extérieur, `x` désigne toujours l'objet 10. L'affichage le prouve : la fonction renvoie 12 et `x` vaut encore 10.
- **Récupérer le résultat.** Comme la fonction ne peut pas changer `x`, c'est le `return` qui transmet la nouvelle valeur, que l'on stocke dans `resultat`.
- `map(fonction, liste)`. Applique la fonction à chaque élément. On passe `ajouter_bonus` **sans parenthèses** : on donne la fonction elle-même, pas son résultat. `map()` n'envoie qu'un argument, donc le `bonus` par défaut (2) est utilisé.
- **Pourquoi `list(...)` ?** `map()` renvoie un objet « paresseux » qui calcule à la demande. `list()` le transforme en vraie liste : `[14.0, 17.5]`. La liste d'origine `notes` n'est pas modifiée.
- `zip()`. Associe les éléments de même position : ("Ali", 14.0) puis ("Sara", 17.5). Dans le `for`, `etudiant`, `note` récupère les deux valeurs de chaque couple. Si les listes n'ont pas la même longueur, `zip()` s'arrête à la plus courte.

Exécution

```
Valeur retournée par la fonction : 12
Valeur de x après l'appel : 10 -> x est resté intact
Notes initiales : [12.0, 15.5]
Notes avec bonus : [14.0, 17.5]
Ali : 14.0 / 20
Sara : 17.5 / 20
```

Exercice 5 — Saisie ordonnée et calcul de la médiane

Notions : listes, `append()`, indices, `while` imbriqué dans un `for`

But. Saisir n notes en imposant l'ordre croissant, les stocker dans une liste, puis calculer la médiane selon que n est pair ou impair.

Solution

```
# === EXERCICE 5 : Saisie ordonnée et calcul de la médiane ===

# 1. Nombre total de notes (au moins 1, sinon la médiane n'existe pas)
n = int(input("Nombre total de notes à traiter : "))
while n <= 0:
    n = int(input("Erreur : il faut au moins une note. Nombre total de notes : "))

# 2. Saisie des notes par ordre croissant
notes = []
for i in range(n):
    note = float(input(f"Note n°{i + 1} : "))
    # À partir de la 2e note, on refuse toute note inférieure à la précédente
    while i > 0 and note < notes[-1]:
        note = float(input(f"Erreur : la note doit être >= {notes[-1]}. Note n°{i + 1} : "))
    notes.append(note)

print("Notes saisies (triées) :", notes)

# 3. et 4. Parité du nombre de notes, puis médiane
milieu = n // 2
if n % 2 == 0:
    # Pair : moyenne des deux valeurs centrales
    print(f"Le nombre de notes ({n}) est pair.")
    mediane = (notes[milieu - 1] + notes[milieu]) / 2
else:
    # Impair : valeur centrale
    print(f"Le nombre de notes ({n}) est impair.")
    mediane = notes[milieu]

print("La médiane est :", mediane)
```

Explication

- **Nombre de notes.** Converti en `int` car il sert à `range(n)`. Une petite boucle `while` refuse 0 ou un nombre négatif : sans note, la médiane n'existe pas.
- **La boucle `for`.** `range(n)` donne $i = 0, 1, \dots, n - 1$: exactement n tours. On affiche $i + 1$ pour numéroté les notes à partir de 1.
- **Le `while` de contrôle.** `notes[-1]` est la dernière note déjà acceptée. Tant que la nouvelle note lui est inférieure, on la redemande. Deux notes égales sont acceptées (ordre croissant au sens large).
- **Pourquoi `i > 0 and` ?** Pour la première note, la liste est vide et `notes[-1]` provoquerait une `IndexError`. Avec `and`, Python n'évalue pas la partie droite quand la partie gauche est fausse : la première note est donc acceptée sans comparaison.
- **`append()`.** La note n'est ajoutée à la liste qu'une fois validée. La liste est donc triée dès la saisie, sans appeler `sort()`.
- **La médiane.** $n // 2$ est la division entière. Si n est impair, la médiane est l'élément central. Si n est pair, il n'y a pas d'élément central : on prend la moyenne des deux du milieu.

n	Liste (indices)	Calcul	Médiane
5 (impair)	[8, 10, 12, 15, 18] — indices 0 à 4	<code>notes[5 // 2] = notes[2]</code>	12
4 (pair)	[8, 12.5, 14, 17] — indices 0 à 3	<code>(notes[1] + notes[2]) / 2</code>	13.25

Exécution

```
Nombre total de notes à traiter : 4
Note n°1 : 8
Note n°2 : 12.5
Note n°3 : 10
Erreur : la note doit être >= 12.5. Note n°3 : 14
Note n°4 : 17
Notes saisies (triées) : [8.0, 12.5, 14.0, 17.0]
Le nombre de notes (4) est pair.
La médiane est : 13.25
```

La note 10 est refusée car elle est inférieure à 12.5 ; l'utilisateur saisit alors 14. Avec 4 notes (nombre pair), la médiane est $(12,5 + 14) / 2 = \mathbf{13,25}$.

Exercice 6 — Configuration d'un système multi-agents

Notions : dictionnaires, listes, tuples, structures imbriquées

But. Construire un dictionnaire qui contient un dictionnaire, qui contient une liste de tuples ; modifier une valeur précise ; puis extraire un seul élément tout au fond de la structure.

Solution

```
# === EXERCICE 6 : Configuration d'un système multi-agents ===

# 1. Dictionnaire principal de configuration
config_ia = {}

# 2. Ajout de l'agent : un dictionnaire imbriqué, contenant une liste de tuples
config_ia["agent_recherche"] = {
    "modele": "gpt-4",
    "temperature": 0.2,
    "outils": [("WebSearch", "v2.1"), ("Calculatrice", "v1.0")],
}
print("Configuration initiale :", config_ia)

# 3. Modification ciblée de la température (sans redéclarer le dictionnaire)
config_ia["agent_recherche"]["temperature"] = 0.5
print("Nouvelle température :", config_ia["agent_recherche"]["temperature"])

# 4. Navigation : dictionnaire -> dictionnaire -> liste (indice 1) -> tuple (indice 0)
print(config_ia["agent_recherche"]["outils"][1][0])
```

Explication

- **Création.** `config_ia = {}` crée un dictionnaire vide. Affecter une valeur à une clé qui n'existe pas encore (`config_ia["agent_recherche"] = ...`) **ajoute** cette clé.
- **Trois structures, trois rôles.** Le dictionnaire donne un nom à chaque paramètre ; la liste contient les outils (on peut en ajouter ou en retirer) ; chaque tuple fige le couple (nom, version) qui ne doit pas changer.
- **Modification.** Un dictionnaire est mutable. On enchaîne les clés pour atteindre la valeur : `config_ia["agent_recherche"]["temperature"] = 0.5`. Seule cette valeur change, le reste du dictionnaire est conservé.

- **Navigation.** Chaque paire de crochets descend d'un niveau, de gauche à droite :

Expression	Ce qu'elle renvoie	Type
<code>config_ia["agent_recherche"]</code>	la configuration de l'agent	dictionnaire
<code>...["outils"]</code>	<code>[("WebSearch", "v2.1"), ("Calculatrice", "v1.0")]</code>	liste
<code>...["outils"][1]</code>	<code>("Calculatrice", "v1.0")</code> — 2 ^e élément, indice 1	tuple
<code>...["outils"][1][0]</code>	<code>"Calculatrice"</code> — 1 ^{er} élément du tuple, indice 0	chaîne

Exécution

```
Configuration initiale : {'agent_recherche': {'modele': 'gpt-4', 'temperature': 0.2, 'outils':
    [ ('WebSearch', 'v2.1'), ('Calculatrice', 'v1.0') ]}}
```

```
Nouvelle température : 0.5
```

```
Calculatrice
```

Piège. Les indices commencent à 0 : « Calculatrice » est le deuxième outil, donc à l'indice **1**. Et un tuple est immuable : `config_ia["agent_recherche"]["outils"][1][0] = "X"` lève une `TypeError`.

Récapitulatif des notions du TP

Notion	À retenir
Conversions	<code>input()</code> renvoie un <code>str</code> ; <code>int()</code> , <code>float()</code> , <code>str()</code> changent le type.
Conditions	<code>if</code> / <code>elif</code> / <code>else</code> ; l'indentation décide de ce qui est imbriqué ou séparé.
Boucles	<code>while</code> quand on ne connaît pas le nombre de tours, <code>for + range()</code> sinon (fin exclue).
Fonctions	paramètres par défaut ; les types de base sont immuables, on récupère le résultat avec <code>return</code> .
Listes	mutables, indexées à partir de 0, <code>[-1]</code> pour le dernier élément, <code>append()</code> pour ajouter.
Tuples / dictionnaires	tuple = groupe figé ; dictionnaire = accès par clé ; ils s'imbriquent librement.
Environnement	<code>uv add</code> pour les dépendances, <code>uv run</code> pour exécuter, <code>uv.lock</code> pour la reproductibilité.